



CORE JAVA INTERVIEW

1. What is Object-Oriented Programming (OOP)?

Cross Q1: Which OOP principle promotes code reusability?

Cross Q2: How is abstraction implemented in Java?

2. What is the difference between `==` and `.equals()`?

Cross Q1: What happens if you use `==` on two String objects with the same value?

Cross Q2: How can you override `.equals()` in your class?

3. What is encapsulation in Java?

Cross Q1: How do private fields and public methods help with encapsulation?

Cross Q2: Can you break encapsulation using reflection?

4. What is inheritance?

Cross Q1: Can a class extend multiple classes?

Cross Q2: How does super keyword help in inheritance?

5. What is polymorphism?

Cross Q1: Difference between method overloading and overriding?

Cross Q2: How is dynamic method dispatch implemented?

6. What is abstraction in Java?

Cross Q1: How are abstract classes and interfaces different?

Cross Q2: Can you declare a constructor in an abstract class?

7. What are the access specifiers in Java?

Cross Q1: What is the default access modifier in Java?

Cross Q2: What is the difference between protected and private?

8. What is a constructor?

Cross Q1: Can a constructor be private? Why use it?

Cross Q2: How is constructor overloading different from method overloading?

9. What is the difference between static and non-static members?

Cross Q1: Can a static method access instance variables?

Cross Q2: Where is static data stored in memory?

10. What is the final keyword in Java?



Cross Q1: What happens if you declare a method as final?

Cross Q2: Can you initialize a final variable inside a constructor?

11. What is the difference between this and super?

Cross Q1: How is this() used in constructor chaining?

Cross Q2: Can super() and this() be used together?

12. What is the role of the main() method in Java?

Cross Q1: Why must main() be static?

Cross Q2: Can we overload main()? Will it be called?

13. What is a package in Java?

Cross Q1: How do you import a package?

Cross Q2: What is the use of java.lang package?

14. What are wrapper classes in Java?

Cross Q1: Why do we need wrapper classes?

Cross Q2: How does autoboxing work internally?

15. What is exception handling in Java?

Cross Q1: Difference between checked and unchecked exceptions?

Cross Q2: Can finally block be skipped?

16. What is the difference between throw and throws?

Cross Q1: Can we write multiple exceptions in a throws clause?

Cross Q2: Where would you use throw in real-time scenarios?

17. What is the use of try-catch-finally blocks?

Cross Q1: What happens if an exception occurs in catch block?

Cross Q2: Can finally execute even without catch?

18. What is multithreading in Java?

Cross Q1: How do you create a thread in Java?

Cross Q2: What is the difference between Runnable and Thread?

19. What is synchronization in Java?

Cross Q1: How does synchronized block work?

Cross Q2: What is a race condition? How do you prevent it?

20. What is the difference between String, StringBuilder, and StringBuffer?



Cross Q1: Which is thread-safe and which is faster?
Cross Q2: How does immutability of String help in security?

21. What is the use of instanceof operator?
Cross Q1: How does it help with safe downcasting?
Cross Q2: Can you override instanceof behavior?

22. What are Java Collections?
Cross Q1: Difference between ArrayList and LinkedList?
Cross Q2: Why is HashMap faster than Hashtable?

23. What is the difference between Array and ArrayList?
Cross Q1: Can you increase the size of an array?
Cross Q2: How does ArrayList work internally?

24. What is garbage collection in Java?
Cross Q1: How do you suggest garbage collection to the JVM?
Cross Q2: What are strong, weak, and soft references?

25. How is memory managed in Java?
Cross Q1: What is the role of stack and heap?
Cross Q2: Where are static and local variables stored?



Core Java Interview Problem Statements Basic Level (OOPs & Fundamentals)

Class & Object:

Write a program to create a Student class with attributes (id, name, marks). Create objects and display details.

Encapsulation:

Create a BankAccount class with private balance. Provide deposit, withdraw, and balance check methods using getters and setters.

Constructor Overloading:

Create a Book class with overloaded constructors (default, parameterized). Show usage.

Method Overloading:

Write a program for a Calculator class with overloaded methods for add(int,int), add(double,double), and add(int,int,int).



Intermediate Level (Inheritance, Polymorphism, Abstraction, Interfaces)

Inheritance & Method Overriding:

Create a Vehicle superclass and Car, Bike subclasses. Override displayInfo() method.

Polymorphism (Runtime):

Create an abstract class Shape with method area(). Extend it into Circle and Rectangle classes. Demonstrate runtime polymorphism.

Interface:

Create an interface Playable with method play(). Implement it in Cricketer and Footballer classes.

Abstract Class & Interface Together:

Create an abstract class Employee with abstract method calculateSalary(). Implement interface Bonus with method getBonus(). Create subclass Manager to implement both.



Advanced Level (Exception Handling, Collections, Strings, File Handling)

Exception Handling:

Write a program to take two integers from user and divide them. Handle ArithmeticException and InputMismatchException.

Custom Exception:

Create a custom exception InsufficientFundsException. Throw it when withdrawal exceeds bank balance.

ArrayList:

Write a program to store Student objects in an ArrayList and display them using a loop.

HashMap:

Create a HashMap<Integer,String> to store roll numbers and student names. Demonstrate insertion, deletion, and search.

String Handling:

Write a program to check whether a string is a palindrome without using built-in reverse function.

File Handling:

Write a program to read student details from a text file and display them on the console.

Multi-threading:

Create two threads: one for printing even numbers and another for printing odd numbers up to 20.

=====



Scenario-Based

Real-World Abstraction:

Model a simple e-commerce system with classes Product, Customer, and Order. Use abstraction to calculate total bill.

Collections with Sorting:

Store employees in a List<Employee> with fields (id, name, salary). Sort by salary using Comparator.

Search Algorithm (Core + Logic):

Implement a program to search a student by roll number from an array using both linear and binary search.

Mini Project (Integration):

Build a mini “Library Management System” with options: add book, issue book, return book, and view all books. Use OOP concepts + Collections.

Factory Design Pattern:

Implement a ShapeFactory that returns objects of Circle, Rectangle, or Square based on user choice.

=====